RolaSoft Technologies. Limited

python™



Web Development on DJANGO

Certificate in Python Django Web Application Developer

COMPUTER & WEB FUNDAMENTALS

COMPUTER & WEB FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

COMPUTER & WEB FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

PYTHON PROGRAMMING FUNDAMENTALS

PYTHON PROGRAMMING FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

PYTHON PROGRAMMING FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Django project & app structure

❓ What is Django Architecture?

Django follows **MTV architecture**:

- **Model** → Database structure
- **Template** → Frontend (HTML)
- **View** → Business logic (connects Model and Template)

MTV is Django's version of **MVC**.

Django project & app structure

🏗️ What is a Django Project?

A project is the main container for your entire web application.

It contains:

- Global settings
- Main URL configuration
- WSGI/ASGI configuration
- Multiple apps

Think of a project like:

🏢 A University

And **apps** are departments inside the university.

Django project & app structure

🎁 What is a Django App

An app is a self-contained module that does one specific job.

Examples:

- Students app
- Blog app
- Accounts app
- Payments app

Each app can be reused in another project.

Django project & app structure

🛠 **STEP 1: Install Django**

pip install Django

Check version:

django-admin --version

🏗 **STEP 2: Create a Django Project**

django-admin startproject myproject

Now go inside:

cd myproject

Run server:

python manage.py runserver

Open browser:

http://127.0.0.1:8000/

You will see the Django welcome page ✓

Django project & app structure

📂 Django Project Structure Explained

After creating a project, your **folder** looks like this:

```
myproject/
│
├── manage.py
│
└── myproject/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Django project & app structure

Let's explain each file carefully 👇

◆ **manage.py**

📌 Command-line utility for interacting with Django.

Examples:

python manage.py runserver

python manage.py makemigrations

python manage.py migrate

python manage.py createsuperuser

Think of it as the remote control of your project.

◆ **init.py**

This tells Python:

"This folder is a Python package."

Usually empty.

Django project & app structure

◆ **settings.py (VERY IMPORTANT)**

This is the brain of your project.

It contains:

- Database configuration
- Installed apps
- Middleware
- Templates configuration
- Static files
- Secret key
- Debug mode

Example:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
]
```

When you create a new app, you must register it here.

Django project & app structure

- **urls.py**

This **file** handles **routing** (like traffic control).

Example:

```
from django.urls import path
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello Students")

urlpatterns = [
    path('', home),
]
```

When user visits /, Django calls home function

Django project & app structure

◆ **wsgi.py & asgi.py**

Used for deployment.

- **WSGI** → Traditional servers (Gunicorn, etc.)
- **ASGI** → Async support (WebSockets, Channels)

You rarely modify these during development.

Django project & app structure

🏗️ STEP 4: Create a Django App

Now we create an app inside the project.

python manage.py startapp students

New structure:

```
myproject/
│
├── manage.py
├── myproject/
│
└── students/
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── views.py
```

Django project & app structure

📂 **Django App Structure Explained**

Let’s explain each file 👇

◆ **admin.py**

Used to register models in Django Admin panel.

Example:

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

Django project & app structure

- **apps.py**

App configuration file.

Contains:

```
from django.apps import AppConfig

class StudentsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'students'
```

Django project & app structure

- **models.py**

Defines database tables.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return self.name
```

Each class = one database table.

Django project & app structure

- **views.py**

Contains logic that processes requests.

Example:

```
from django.http import HttpResponse

def student_list(request):
    return HttpResponse("List of students")
```

Django project & app structure

◆ **migrations/**

Stores database migration files.

When you run:

python manage.py makemigrations

python manage.py migrate

Django creates **files** inside this **folder**.

◆ **tests.py**

Used for writing automated tests.

Django project & app structure

🔄 **Connecting App to Project (VERY IMPORTANT)**

After creating an app, you MUST:

Step 1: Register app in settings.py

```
INSTALLED_APPS = [
    ...
    'students',
]
```

Step 2: Create app URLs

Create students/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.student_list, name='student_list'),
]
```

Django project & app structure

Step 3: Connect App URLs to Project URLs

In project urls.py:

```
from django.urls import path, include

urlpatterns = [
    path('students/', include('students.urls')),
]
```

Now visit:

http://127.0.0.1:8000/students/

Django project & app structure

🏢 **Final Structure (Professional Setup)**

```
myproject/
│
├── manage.py
│
├── myproject/
│   ├── settings.py
│   ├── urls.py
│
└── students/
    ├── models.py
    ├── views.py
    ├── urls.py
    ├── templates/
    ├── static/
```

Django project & app structure

🎯 **Real-World Best Practice**

In large systems:

- One app = One responsibility
- Never put all logic in one app
- Keep project clean and modular

Example:

accounts/
students/
payments/
reports/

Django project & app structure

Key Commands

django-admin startproject projectname
python manage.py startapp appname
python manage.py runserver
python manage.py makemigrations
python manage.py migrate

Django project & app structure

📁 FINAL PROJECT STRUCTURE

```
rolasoft_school/
│
├── manage.py
│
├── rolasoft_school/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
└── students/
    ├── migrations/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── views.py
    └── urls.py
```

LAB 1 ACTIVITY -Django Activity

🏆 **Lab 1 – Activity**

1. Create project called rolasoft_school
2. Create app called students
3. Add a simple view returning "Welcome to Rolasoft Students Portal"
4. Connect URLs properly
5. Run server and test

LAB 2 ACTIVITY - Login System (Authentication)

🏆 **Lab 2 – Login System (Authentication)**

Students App will support:

- ➕ Create Student
- 📄 View Students
- ✏️ Edit Student
- ❌ Delete Student
- 🔐 Login / Logout
- 🛑 Only logged-in users can access CRUD

Settings & configurations

🔧 Topic: Settings & Configurations

❓ What is settings.py?

settings.py is the central configuration file of your Django project.

It controls:

- Security
- Database
- Installed apps
- Middleware
- Templates
- Static & media files
- Authentication behavior
- Email configuration
- Production vs development behaviour

Think of it as:

❓ The Brain of Your Entire Django Project

Location:

project_name/project_name/settings.py

Settings & configurations

🗁 Basic Structure of settings.py

A typical file looks like this:

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
SECRET_KEY = 'your-secret-key'
DEBUG = True
ALLOWED_HOSTS = []
INSTALLED_APPS = []
MIDDLEWARE = []
TEMPLATES = []
DATABASES = {}
STATIC_URL = 'static/'
```

Now we will break this down professionally.

Settings & configurations

🔐 **SECRET_KEY (Security Core)**

SECRET_KEY = 'django-insecure-abc123'

This key is used for:

- Password hashing
- Sessions
- CSRF protection
- Security tokens

⚠ Never expose it publicly

⚠ Never push real secret keys to GitHub

Settings & configurations

DEBUG Mode

DEBUG = True

Development:

DEBUG = True

Production:

DEBUG = False

If DEBUG = False:

- Error pages become hidden
- ALLOWED_HOSTS must be set
- Static files must be configured properly

Settings & configurations

🌍 **ALLOWED_HOSTS**

ALLOWED_HOSTS = []

In production:

ALLOWED_HOSTS = ['yourdomain.com']

Prevents host header attacks.

Settings & configurations

🎁 **INSTALLED_APPS**

This tells Django which apps are active.

Default:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

When you create an app:

python manage.py startapp students

You must add:

```
INSTALLED_APPS = [
    ...
    'students',
]
```

If not added → Django will ignore your app.

Settings & configurations

MIDDLEWARE

Middleware runs between request and response.

Default example:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]
```

Middleware handles:

- Security
- Sessions
- Authentication
- CSRF protection
- Clickjacking protection

Settings & configurations

🗄 **DATABASE Configuration**

Default SQLite:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Settings & configurations

● **Using PostgreSQL (Production Example)**

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'school_db',
        'USER': 'postgres',
        'PASSWORD': 'password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}
```

After editing → run:

python manage.py migrate

Settings & configurations

🎨 **TEMPLATES Configuration**

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

Important Parts:

- **DIRS** → Global templates folder
- **APP_DIRS = True** → Enables app-level templates

Settings & configurations

🖼 Static Files Configuration

Static files = CSS, JS, Images

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
STATIC_ROOT = BASE_DIR / "staticfiles"
```

Development:

Served automatically.

Production:

Run:

```
python manage.py collectstatic
```

Settings & configurations

📁 Media Files (Uploads)

For user uploads:

MEDIA_URL = '/media/'

MEDIA_ROOT = BASE_DIR / 'media'

Add to urls.py (development only):

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Settings & configurations

🌎 **Internationalization (Time & Language)**

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

Example for Nigeria:

TIME_ZONE = 'Africa/Lagos'

🔐 **Authentication Settings**

LOGIN_URL = 'login'

LOGIN_REDIRECT_URL = 'home'

LOGOUT_REDIRECT_URL = 'login'

These control login behavior.

Settings & configurations

📧 Email Configuration

Example using Gmail:

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'your_email@gmail.com'
EMAIL_HOST_PASSWORD = 'your_app_password'
```

🏭 Production Best Practice (Split Settings)

Large projects split settings into:

```
settings/
    __init__.py
    base.py
    development.py
    production.py
```

This is professional structure.

Settings & configurations

🔒 **Environment Variables (VERY IMPORTANT)**

Never hardcode secrets.

Install:

pip install python-decouple

Use:

from decouple import config

SECRET_KEY = config('SECRET_KEY')

DEBUG = config('DEBUG', default=False, cast=bool)

Then create .env file:

SECRET_KEY=supersecretkey

DEBUG=True

Settings & configurations

⍰ How Settings Work Internally

When Django starts:

1. It loads settings.py
2. Configures database
3. Loads installed apps
4. Registers middleware
5. Prepares template engine
6. Starts server

LAB 3 ACTIVITY - Modify your project

🏆 **Lab 3 – Modify your project**

Modify your project to:

1. Change TIME_ZONE to Africa/Lagos
2. Add global templates folder
3. Configure media upload
4. Configure login redirect
5. Split settings into development & production

URLs & views

🌐 **Topic: URLs & Views**

How Django Handles a Request

When a user types:

http://127.0.0.1:8000/students/

Django follows this flow:

1. Browser sends request
2. Django checks urls.py
3. URL pattern matches
4. Corresponding view function runs
5. View returns response
6. Browser displays result

This is the heart of Django.

URLs & views

🧰 Setting Up Example Project

If not already created:

```
django-admin startproject school_project
cd school_project
python manage.py startapp students
```

Register app in settings.py:

```
INSTALLED_APPS = [
    ...
    'students',
]
```

🌐 Understanding URL Configuration

Django uses a URL dispatcher.

There are two levels:

1. Project-level URLs
2. App-level URLs

URLs & views

🏛 **Project-Level URLs**

Open:

school_project/urls.py

Default content:

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

This means:

- When user visits /admin/, open admin panel.

URLs & views

🔗 Connecting App URLs (Best Practice)

Instead of putting all routes in project file, we delegate to apps.

Modify project urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]
```

Now Django will look inside:

students/urls.py

URLs & views

📂 Create App URLs File

Inside students folder, create:

students/urls.py

Add:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

Explanation:

- '' → homepage
- views.home → function inside views.py
- name='home' → URL name for referencing

URLs & views

👤💻 Creating Views

Open:
students/views.py

Example 1: Simple Response

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to Rolasoft School")
```

Now visit:
http://127.0.0.1:8000/

You will see:
Welcome to Rolasoft School

URLs & views

🎨 Rendering Templates in Views

Instead of returning plain text, we use templates.

Step 1: Create Template Folder

```
students/
└── templates/
    └── students/
        └── home.html
```

Step 2: Create home.html

```
<h1>Welcome to Rolasoft Students Portal</h1>
```

Step 3: Update View

```
from django.shortcuts import render
def home(request):
    return render(request, 'students/home.html')
```

Now Django renders HTML file.

URLs & views

🔄 URL Patterns Explained

Basic syntax:

path(route, view, name)

Example:

path('about/', views.about, name='about')

This means:

- URL: /about/
- Calls: about() view

URLs & views

🔢 Dynamic URLs (Passing Parameters)

Example:

User visits:

/student/5/

In urls.py

```
path('student/<int:id>/', views.student_detail, name='student_detail')
```

In views.py

```
def student_detail(request, id):
    return HttpResponse(f"Student ID is {id}")
```

If user visits:

/student/10/

Output:

Student ID is 10

URLs & views

? Types of URL Converters

Type	Example
int	<int:id>
str	<str:name>
slug	<slug:slug>
uuid	<uuid:id>
path	<path:filepath>

Example:

path('course/<slug:slug>/', views.course_detail)

URLs & views

🔁 **Reverse URL (Very Important)**

Instead of hardcoding URLs:

❌ **Bad:**

```
<a href="/student/5/">
```

✓ **Good:**

```
<a href="{% url 'student_detail' 5 %}">
```

This uses the name from urls.py.

URLs & views

□ Multiple Views Example

In views.py:

```
def about(request):
    return HttpResponse("About Page")

def contact(request):
    return HttpResponse("Contact Page")
```

In urls.py:

```
urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
    path('contact/', views.contact, name='contact'),
]
```

URLs & views

⚡ HTTP Methods in Views

Django views can handle:

- GET
- POST
- PUT
- DELETE

Example:

```
def example(request):
    if request.method == "POST":
        return HttpResponse("Form Submitted")
    return HttpResponse("Normal Page Load")
```

□ Function-Based Views (FBVs) vs Class-Based Views (CBVs)

Today we focus on FBVs.

Basic FBV structure:

```
def view_name(request):
    # logic
    return response
```

URLs & views

✅ Testing URLs & Views

Run server:

python manage.py runserver

Test these URLs:

/

/about/

/contact/

/student/3/

If correctly configured → each page loads.

URLs & views

🏗️ Real-World Structure

Professional Django apps use:

```
project/
│
├── project/urls.py
│
└── students/
    ├── urls.py
    ├── views.py
    ├── templates/
```

Never place all URLs in one file for large projects.

LAB 4 ACTIVITY

🏆 **Lab 4**

1. Create a page /courses/
2. Create dynamic URL /courses/<int:id>/
3. Render template instead of HttpResponse
4. Add navigation links using {% url %}

Templates & template inheritance

🎨 Templates & Template Inheritance

❓ What Are Templates in Django?

A template is an HTML file that allows dynamic data using Django Template Language (DTL).

Instead of writing:

return HttpResponse("<h1>Hello</h1>")

We use:

return render(request, "home.html")

Templates separate:

- Backend logic (Python)
- Frontend design (HTML)

This follows Django's MTV architecture.

Templates & template inheritance

📁 Setting Up Templates Properly

Step 1: Create Templates Folder

Inside your app:

students/
└── templates/
    └── students/

OR

create global templates folder:

project/
├── templates/

Templates & template inheritance

Step 2: Configure settings.py

Open settings.py and confirm:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],  # Global templates
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

APP_DIRS=True allows app-level templates.

Templates & template inheritance

👤💻 Rendering a Template

In views.py:

```
from django.shortcuts import render


def home(request):
    return render(request, 'students/home.html')
```

In students/templates/students/home.html:

```
<h1>Welcome to Rolasoft School</h1>
```

Run server → It works 🎉

Templates & template inheritance

Django Template Language (DTL)

Django templates use:

- {{ }} → Variables
- {% %} → Logic
- {# #} → Comments

Variables Example

In view:

```
def home(request):
    context = {
        "name": "Rolasoft",
        "year": 2026
    }
    return render(request, 'students/home.html', context)
```

In template:

```
<h1>Welcome to {{ name }}</h1>
<p>Year: {{ year }}</p>
```

Templates & template inheritance

For Loop Example

View:

```
def course_list(request):
    courses = ["Python", "Django", "React"]
    return render(request, "students/course_list.html", {"courses": courses})
```

Template:

```
<ul>
    {% for course in courses %}
        <li>{{ course }}</li>
    {% endfor %}
</ul>
```

If Statement Example

```
{% if courses %}
    <p>Courses Available</p>
{% else %}
    <p>No courses available</p>
{% endif %}
```

Templates & template inheritance

🎨 **Template Inheritance (VERY IMPORTANT)**

Now we move to professional structure.

Instead of repeating:

- Header
- Navbar
- Footer

On every page — we use inheritance.

Templates & template inheritance

📁 Create Base Template

Create in:

templates/base.html

Add:

```
<!DOCTYPE html>
<html>
<head>
   <title>Rolasoft School</title>
</head>
<body>
   <header>
      <h1>Rolasoft Portal</h1>
      <nav>
         <a href="{% url 'home' %}">Home</a> |
         <a href="{% url 'course_list' %}">Courses</a>
      </nav>
      <hr>
   </header>
```

Templates & template inheritance

```
{% block content %}
    {% endblock %}

    <hr>
    <footer>
        <p>&copy; 2026 Rolasoft Technologies</p>
    </footer>

</body>
</html>
```

Notice:

{% block content %}{% endblock %}

This is a placeholder.

Templates & template inheritance

Extend Base Template

Now modify home.html:

```
{% extends "base.html" %}

{% block content %}
<h2>Welcome to Rolasoft School</h2>
<p>This is the homepage.</p>
{% endblock %}
```

Modify course_list.html:

```
{% extends "base.html" %}

{% block content %}
<h2>Available Courses</h2>
```

Templates & template inheritance

```
<ul>
    {% for course in courses %}
        <li>{{ course }}</li>
    {% endfor %}
</ul>
{% endblock %}
```

Now:

- Header
- Navbar
- Footer

Are automatically included.

Templates & template inheritance

How Template Inheritance Works

1. Django loads child template
2. Sees {% extends "base.html" %}
3. Loads base template
4. Replaces {% block content %} with child content

This prevents code repetition.

Multiple Blocks Example

In base.html:

<title>{% block title %}Rolasoft{% endblock %}</title>

In child template:

{% block title %}Home Page{% endblock %}

Now pages can have different titles.

Templates & template inheritance

🎨 Static Files in Templates

To use CSS:

At top of template:

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">
```

Make sure in settings:

```
STATIC_URL = '/static/'
```

🧩 Including Templates

You can include small components.

Example:

Create:

templates/navbar.html

In base.html:

```
{% include "navbar.html" %}
```

Used for reusable components.

Templates & template inheritance

❓ Template Best Practices

✓ Always use template inheritance
✓ Never repeat header/footer
✓ Use named URLs
✓ Keep logic minimal inside templates
✓ Avoid heavy business logic in templates

🏗️ Professional Template Structure

```
templates/
├── base.html
├── navbar.html
│
└── students/
    ├── home.html
    ├── course_list.html
    └── course_detail.html
```

Templates & template inheritance

🏆 LAB-5

Upgrade your project:

1. Add Bootstrap via CDN in base.html
2. Create reusable navbar.html
3. Add page title block
4. Add dynamic footer year using context
5. Style course list with Bootstrap cards

Static & media files

🖼 Topic: Static & Media Files

❓ What Are Static & Media Files?

Static Files:

Files that don’t change dynamically:

- CSS
- JavaScript
- Images
- Fonts

Media Files:

Files uploaded by users at runtime:

- Profile pictures
- Documents
- PDFs
- Any uploaded content

Static & media files

🗂️ Directory Structure

A typical Django project looks like this for static & media:

```
project/
├── static/          # Global static files
│   ├── css/
│   └── js/
├── media/           # User uploaded files
│   └── students/
└── students/
    └── static/students/  # App-level static files
```

Static & media files

⚙️ Configuring Static Files

Open settings.py:

```
STATIC_URL = '/static/'

# Optional: Global static folder
STATICFILES_DIRS = [
    BASE_DIR / "static",
]

# For production (collectstatic)
STATIC_ROOT = BASE_DIR / "staticfiles"
```

- STATIC_URL → URL prefix
- STATICFILES_DIRS → Extra directories to search for static files
- STATIC_ROOT → Where static files are collected for production

Static & media files

□ Using Static Files in Templates

Step 1: Load static tag

{% load static %}

Step 2: Reference CSS/JS/Images

<link rel="stylesheet" href="{% static 'css/style.css' %}">

<img src="{% static 'images/logo.png' %}" alt="Logo">

- Always use {% static %} tag
- Never hardcode /static/

Static & media files

✅ Example: Styling Home Page

Create static/css/style.css:

```
body {
    background-color: #f8f9fa;
}

h1 {
    color: #007bff;
}
```

In home.html:

```
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">

<h1>Welcome to Rolasoft School</h1>
```

Static & media files

Configuring Media Files (User Uploads)

Add in settings.py:

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media‘
```

- MEDIA_URL → URL prefix to access files
- MEDIA_ROOT → Folder on disk to store files

Static & media files

Update urls.py for Media (Development Only)

Open project/urls.py:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('students.urls')),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Now **Django** can serve uploaded files during development.

Static & media files

✅ Example: Upload Image for Students

In students/models.py:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    profile_image = models.ImageField(upload_to='students/', null=True, blank=True)

    def __str__(self):
        return self.name
```

- upload_to='students/' → stored in media/students/

Install Pillow:

```
pip install pillow
```

Static & media files

Display Uploaded Media in Templates

In student_detail.html:

```
<h2>{{ student.name }}</h2>

{% if student.profile_image %}
    <img src="{{ student.profile_image.url }}" alt="{{ student.name }}" width="200">
{% else %}
    <p>No profile image uploaded.</p>
{% endif %}
```

Static & media files

Admin Support for Media

In students/admin.py:

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'profile_image']
```

Upload image directly from admin panel.

Static & media files

Collectstatic for Production

For deployment:

python manage.py collectstatic

- Collects all static files into STATIC_ROOT
- Configure your web server (Nginx/Apache) to serve /static/
- Media files are served from MEDIA_ROOT

Tips & Best Practices

✓ Keep separate global and app-level static folders

✓ Always use {% static %} for references

✓ Serve media files with proper permissions

✓ In production, serve static & media using Nginx/Apache

✓ Never store uploaded files inside static folders

Static & media files

Professional Folder Example

```
rolasoft_school/
├── static/
│   ├── css/
│   ├── js/
│   └── images/
├── media/
│   └── students/
└── students/
    └── static/students/
```

Static & media files

🏆 LAB 6

1. Add a CSS file to style homepage
2. Add a JS file for interactivity (e.g., alert welcome message)
3. Upload profile image for student
4. Display student images in template
5. Test both static & media file rendering

Models & migrations

🏗️ Models & Migrations

🤔 What Are Models in Django?

A model is a Python class that represents a database table.

- **Each attribute** → a table column
- **Each instance** → a row in the table
- Django uses ORM (Object-Relational Mapping) to map Python objects to database tables

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    enrollment_date = models.DateField(auto_now_add=True)
```

- **name** → VARCHAR(100)
- **email** → UNIQUE VARCHAR field
- **enrollment_date** → automatically set when a student is created

Models & migrations

🏗️ Creating Models Step-by-Step

1) Open your app, e.g., students/models.py

2) Define model classes:

```
from django.db import models

class Course(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    start_date = models.DateField()
    def __str__(self):
        return self.title
```

3) Notes:

- **CharField** → short text
- **TextField** → long text
- **DateField** → stores date
- **__str__** → returns readable name in admin panel

Models & migrations

? Field Types You Must Know

Field Type	Example	Notes
CharField	name = models.CharField(max_length=100)	Short text
TextField	desc = models.TextField()	Long text
IntegerField	age = models.IntegerField()	Integer
FloatField	price = models.FloatField()	Decimal number
BooleanField	active = models.BooleanField(default=True)	True/False
DateField	dob = models.DateField()	Date
DateTimeField	created = models.DateTimeField(auto_now_add= True)	Timestamp
EmailField	email = models.EmailField(unique=True)	Email validation
ImageField	photo = models.ImageField(upload_to='students /')	Upload image (requires Pillow)
ForeignKey	course = models.ForeignKey(Course, on_delete=models.CASCADE)	One-to-many relationship

Models & migrations

? What Are Migrations?

Migrations are Django's way of applying changes to the database.

Workflow:

1. **Define models** → models.py
2. **Create migration** → python manage.py makemigrations
3. **Apply migration** → python manage.py migrate
   - Creates database tables automatically
   - Keeps track of changes like adding/removing fields

Models & migrations

Create migrations:

python manage.py makemigrations students

Output example:

Migrations for 'students':
  students/migrations/0001_initial.py
    - Create model Student
    - Create model Course

Apply migrations:

python manage.py migrate

- Tables created in the database
- No manual SQL needed

Models & migrations

Checking Database Schema

Option 1: Django admin
Option 2: SQLite CLI (if using SQLite)
Option 3: Django shell:

```
python manage.py shell
from students.models import Student, Course
Student.objects.all()
Course.objects.all()
```

Models & migrations

? Updating Models

1) Add a new field to Student:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    enrollment_date = models.DateField(auto_now_add=True)
    profile_image = models.ImageField(upload_to='students/', null=True, blank=True)
```

2) Create new migration:

```
python manage.py makemigrations students
```

3) Apply migration:

```
python manage.py migrate
```

Django automatically alters the database.

Models & migrations

□? Relationships Between Models

1) One-to-Many (ForeignKey):

```
class Course(models.Model):
    title = models.CharField(max_length=200)

class Student(models.Model):
    name = models.CharField(max_length=100)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
```

- One course → Many students
- on_delete=models.CASCADE → deleting course deletes students

2) Many-to-Many:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(Course)
```

- **Students** can enroll in multiple courses
- **Courses** can have multiple students

Models & migrations

? Using Models in Views

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students})
```

Template:

```
<ul>
    {% for student in students %}
        <li>{{ student.name }} - {{ student.email }}</li>
    {% endfor %}
</ul>
```

Models & migrations

□ Admin Integration

```
from django.contrib import admin
from .models import Student, Course

admin.site.register(Student)
admin.site.register(Course)
```

- **Create superuser:**

```
python manage.py createsuperuser
```

- **Add and edit records** via Django admin panel

Models & migrations

🧰 Best Practices

✓ Always define __str__() for models

✓ Keep migrations small and incremental

✓ Use null=True, blank=True when field optional

✓ Use related_name for ForeignKey / ManyToManyField

✓ Don’t modify applied migrations manually – create a new one

Models & migrations

🏁 LAB 7 Class Work

1. Create Course model with title & description
2. Create Student model with name, email, enrollment_date
3. Add a profile_image field to Student
4. Create a Many-to-Many relationship: Student ↔ Course
5. Apply migrations and add 3 students + 2 courses via admin
6. Display students & courses in a template

We assume your Django project is **rolasoft_school** and your app is **students**.

Relationships

🔗 Relationships

Django's ORM makes it easy to define relationships between models without writing raw SQL.

❓ Relationship Types in Django

Relationship Type	Django Field	Real World Example
One-to-One	OneToOneField	User ↔ Profile
One-to-Many	ForeignKey	Course → Students
Many-to-Many	ManyToManyField	Students ↔ Courses

Relationships

? One-to-One Relationship

- **Definition**: Each record in one table corresponds to exactly one record in another table.
- **Example**: Each User has exactly one Profile.

```
from django.db import models
from django.contrib.auth.models import User
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField()
    profile_image = models.ImageField(upload_to='profiles/', null=True, blank=True)

    def __str__(self):
        return f"{self.user.username}'s Profile"
```

Notes:

- on_delete=models.CASCADE → deleting the User also deletes the Profile
- Access related object:

```
user = User.objects.first()
profile = user.profile  # reverse lookup
```

Relationships

? One-to-Many Relationship

- **Definition**: One record in a table can relate to multiple records in another table.
- **Example**: One Course has many Students.

```
class Course(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    course = models.ForeignKey(Course, on_delete=models.CASCADE, related_name='students')
    def __str__(self):
        return self.name
```

Notes:

- ForeignKey → defines One-to-Many
- related_name='students' → allows course.students.all()
- Reverse lookup:

```
course = Course.objects.first()
all_students = course.students.all()
```

Relationships

? Many-to-Many Relationship

- **Definition**: Multiple records in one table relate to multiple records in another table.
- **Example**: Students can enroll in multiple courses, and courses can have multiple students.

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    courses = models.ManyToManyField(Course, related_name='students')

    def __str__(self):
        return self.name
```

Notes:

- ManyToManyField automatically creates a join table
- Access related data:

```
student = Student.objects.first()
student_courses = student.courses.all()
course = Course.objects.first()
enrolled_students = course.students.all()
```

Relationships

? Summary of Relationships Syntax

Relationship	Field Type	Example
One-to-One	OneToOneField	user = models.OneToOneField(User)
One-to-Many	ForeignKey	course = models.ForeignKey(Course)
Many-to-Many	ManyToManyField	courses = models.ManyToManyField(Course)

Relationships

? Using Relationships in Views & Templates

Example: Display students with their courses

```
def student_list(request):
    students = Student.objects.all().prefetch_related('courses')
    return render(request, 'students/student_list.html', {'students': students})
```

Template:

```
{% for student in students %}
    <h3>{{ student.name }}</h3>
    <p>Courses:</p>
    <ul>
        {% for course in student.courses.all %}
            <li>{{ course.title }}</li>
        {% empty %}
            <li>No courses enrolled</li>
        {% endfor %}
    </ul>
{% endfor %}
```

Relationships

□? Admin Setup for Relationships

```
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'email']
    filter_horizontal = ['courses']  # Easy selection for Many-to-Many
```

- **filter_horizontal** → useful for Many-to-Many
- **related_name** → simplifies reverse lookups

Relationships

? Practical Example: Combining All Three

```
from django.contrib.auth.models import User

class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField()

class Course(models.Model):
    title = models.CharField(max_length=200)

class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    courses = models.ManyToManyField(Course, related_name='students')
```

- Each student has one user account
- Each student can enroll in multiple courses
- Each course can have multiple students

Relationships

🏁 Best Practices

✓ Always use related_name for ForeignKey / ManyToManyField

✓ One-to-One → use for profiles or settings linked to a user

✓ One-to-Many → default choice for hierarchical data (like courses → students)

✓ Many-to-Many → use join tables automatically handled by Django

✓ Prefetch related objects for performance

Relationships

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Relationships

🏆 LAB 8: Class Work

1. Create Profile → One-to-One with User
2. Create Course → One-to-Many with Student
3. Create Student → Many-to-Many with Course
4. Display in template:
   - User → Profile info
   - Student → Courses
   - Course → Enrolled Students

We assume your Django project is **rolasoft_school** and your app is **students**.

Django ORM queries

🧰 Django ORM Queries

Django ORM allows you to interact with your database using Python code, without writing SQL directly.

🔍 Getting All Objects

from students.models import Student, Course

Get all students

students = Student.objects.all()

Get all courses

courses = Course.objects.all()

- all() → returns QuerySet of all records
- QuerySets are lazy → evaluated only when needed

Django ORM queries

🔍 Getting Single Object

Get a student by ID

student = Student.objects.get(id=1)

Get a course by title

course = Course.objects.get(title="Python Programming")

- get() → returns one object
- Throws DoesNotExist if not found, MultipleObjectsReturned if multiple match

✅ Use get() only when you are sure there’s exactly one match

Django ORM queries

🔍 Filtering Objects

```
# Filter students enrolled in a course with ID 1
students_in_course = Student.objects.filter(courses__id=1)

# Filter students by name containing "Alice"
students_named_alice = Student.objects.filter(user__username__icontains="Alice")

# Multiple filters
students_filtered = Student.objects.filter(user__username__icontains="Alice",
enrollment_date__year=2026)
```

- filter() → returns QuerySet
- Double underscore (__) allows lookups and relationships
- icontains → case-insensitive search

Django ORM queries

□ Excluding Objects

Students NOT enrolled in course 1

students_not_in_course = Student.objects.exclude(courses__id=1)

- exclude() → returns all objects except those matching the criteria

□ Ordering Objects

Order students by enrollment date ascending

students_ordered = Student.objects.all().order_by('enrollment_date')

Order by descending

students_ordered_desc = Student.objects.all().order_by('-enrollment_date')

- order_by() → sorts results
- Prefix - → descending order

Django ORM queries

✅ Limiting & Slicing

```
# Get first 5 students
first_five_students = Student.objects.all()[:5]

# Skip first 3 and get next 2
some_students = Student.objects.all()[3:5]
```

- Django QuerySets support slicing like lists

✅ Using Relationships in Queries

```
# Get all courses for a student
student = Student.objects.get(id=1)
courses = student.courses.all()
# Get all students in a course
course = Course.objects.get(id=1)
enrolled_students = course.students.all()
# Filter students whose username contains 'Alice' AND enrolled in 'Python Programming'
students_filtered = Student.objects.filter(user__username__icontains='Alice', courses__title='Python Programming')
```

- Use __ to traverse relationships: ForeignKey, OneToOne, ManyToMany

Django ORM queries

? Aggregation & Counting

```
from django.db.models import Count, Avg

# Count total students
total_students = Student.objects.count()

# Count students in each course
course_counts = Course.objects.annotate(num_students=Count('students'))
for c in course_counts:
    print(c.title, c.num_students)

# Average number of courses per student
average_courses =
Student.objects.annotate(num_courses=Count('courses')).aggregate(Avg('num_courses'))
```

- **annotate()** → adds calculated fields
- **aggregate()** → calculates overall summary

Django ORM queries

? Complex Queries with Q Objects

```
from django.db.models import Q
# Students whose name is Alice OR enrolled in Python course
students_complex = Student.objects.filter(
    Q(user__username='Alice') | Q(courses__title='Python Programming')
)


# NOT condition
students_not_alice = Student.objects.filter(~Q(user__username='Alice'))
```

- Q → allows OR / NOT conditions in queries

? Selecting Specific Fields

```
# Only get usernames of students
usernames = Student.objects.values('user__username')
# Get a list of tuples
student_list = Student.objects.values_list('user__username', 'enrollment_date')
```

- values() → returns dicts
- values_list() → returns tuples

Django ORM queries

⁉ Prefetching / Optimizing Queries

Avoid multiple queries for Many-to-Many

students = Student.objects.prefetch_related('courses').all()

For ForeignKey / One-to-One

students = Student.objects.select_related('user').all()

- select_related → follows ForeignKey/OneToOne (single join)
- prefetch_related → follows Many-to-Many (efficient batch query)

✓ Always use these to avoid the N+1 query problem in templates

⁉ Deleting Objects

Delete a student

student = Student.objects.get(id=1)

student.delete()

Delete all courses with no students

Course.objects.annotate(num_students=Count('students')).filter(num_students=0).delete()

Django ORM queries

? Summary Table

Operation	ORM Method	Example
All objects	.all()	Student.objects.all()
Filter	.filter()	Student.objects.filter(user__username='Alice')
Exclude	.exclude()	Student.objects.exclude(courses__id=1)
Get single object	.get()	Student.objects.get(id=1)
Order	.order_by()	Student.objects.order_by('-enrollment_date')
Count	.count()	Student.objects.count()
Aggregate	.aggregate() / .annotate()	Course.objects.annotate(num_students=Count('students'))
Complex conditions	Q()	`Student.objects.filter(Q(user__username='Alice')
Related objects	.select_related() / .prefetch_related()	Student.objects.prefetch_related('courses')
Delete	.delete()	student.delete()

Django ORM queries

□ Example: Display in Template

```
<h2>Students & Courses</h2>
<ul>
  {% for student in students %}
    <li>
      {{ student.user.username }} - Courses:
      {% for course in student.courses.all %}
        {{ course.title }}{% if not forloop.last %}, {% endif %}
      {% empty %}
        No courses
      {% endfor %}
    </li>
  {% endfor %}
</ul>
```

- Always use select_related / prefetch_related to optimize queries

Django ORM queries

🏆 Key Takeaways

1. Django ORM allows you to query data efficiently in Python.
2. Use filter, exclude, get, order_by, annotate, aggregate for most tasks.
3. Use Q objects for complex OR / NOT conditions.
4. Use select_related and prefetch_related to avoid extra database queries.
5. Templates should iterate over QuerySets efficiently with related objects.

Django ORM queries

🏆 LAB 9: Class Work

Django ORM queries in action, with:

- Models (User, Profile, Student, Course)
- Admin setup
- Views using ORM queries (all, filter, exclude, annotate, aggregate, select_related, prefetch_related)
- Templates showing query results
- Pre-filled sample data (via fixture)
- Navbar + Bootstrap
- Static & media files

Django forms

🧰 Django Forms

Django provides two main types of forms:

1. **forms.Form** → for custom forms not tied to models
2. **forms.ModelForm** → for forms that are directly linked to Django models

Django forms

Creating a Simple Form (forms.Form)

Suppose we want a contact form:

students/forms.py

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100, label="Your Name")
    email = forms.EmailField(label="Email Address")
    message = forms.CharField(widget=forms.Textarea, label="Message")
```

Key points:

- CharField → single-line text
- EmailField → validates email automatically
- widget=forms.Textarea → multi-line input
- label → custom field label

Django forms

Using the Form in Views

students/views.py

```
from django.shortcuts import render
from .forms import ContactForm

def contact(request):
    if request.method == 'POST':
        form = ContactForm(request.POST)
        if form.is_valid():
            # Access cleaned data
            name = form.cleaned_data['name']
            email = form.cleaned_data['email']
            message = form.cleaned_data['message']
```

Django forms

```
            # For demonstration, just print
            print(f"Received message from {name} ({email}): {message}")
            success = True
        else:
            success = False
    else:
        form = ContactForm()
        success = None

    return render(request, 'students/contact.html', {'form': form, 'success': success})
```

Notes:

- request.POST → contains submitted data
- form.is_valid() → runs validation
- form.cleaned_data → safe, validated data

Django forms

✅ Template for Form (students/templates/students/contact.html)

```
{% extends "base.html" %}
{% block title %}Contact{% endblock %}
{% block content %}
<h2>Contact Form</h2>
{% if success == True %}
  <div class="alert alert-success">Your message has been sent!</div>
{% elif success == False %}
  <div class="alert alert-danger">Please correct the errors below.</div>
{% endif %}

<form method="post" novalidate>
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit" class="btn btn-primary">Send</button>
</form>
{% endblock %}
```

- {{ form.as_p }} → renders fields as <p> tags
- csrf_token → required for security

Django forms

□ Model Forms (forms.ModelForm)

Suppose we have a Student model:

```
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['user', 'courses']
```

Key points:

- Automatically generates form fields from model fields
- Handles validation based on model constraints
- Supports saving data directly to the database

Django forms

✅ Using ModelForm in Views

```
def add_student(request):
    if request.method == 'POST':
        form = StudentForm(request.POST)
        if form.is_valid():
            form.save()
            success = True
        else:
            success = False
    else:
        form = StudentForm()
        success = None

    return render(request, 'students/add_student.html', {'form': form, 'success': success})
```

Django forms

✅ Template for ModelForm (students/templates/students/add_student.html)

```
{% extends "base.html" %}
{% block title %}Add Student{% endblock %}
{% block content %}
<h2>Add Student</h2>

{% if success == True %}
  <div class="alert alert-success">Student added successfully!</div>
{% elif success == False %}
  <div class="alert alert-danger">Fix the errors below.</div>
{% endif %}

<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit" class="btn btn-success">Add Student</button>
</form>
{% endblock %}
```

Django forms

✅ Form Validation

Django handles:

- Required fields → required=True (default)
- **Max/min length** → max_length in CharField
- **Email validation** → EmailField
- Custom validation

Custom validation example:

```
class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)

    def clean_name(self):
        name = self.cleaned_data['name']
        if any(char.isdigit() for char in name):
            raise forms.ValidationError("Name cannot contain numbers")
        return name
```

Django forms

? File Uploads in Forms

Suppose we want students to upload a profile image:

```
class StudentUploadForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['user', 'profile_image', 'courses']
```

View:

```
def upload_student(request):
    if request.method == 'POST':
        form = StudentUploadForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            success = True
        else:
            success = False
    else:
        form = StudentUploadForm()
        success = None
    return render(request, 'students/upload_student.html', {'form': form, 'success': success})
```

Django forms

Template:

```
<form method="post" enctype="multipart/form-data">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit" class="btn btn-primary">Upload</button>
</form>
```

- enctype="multipart/form-data" → required for file uploads
- request.FILES → contains uploaded files

Django forms

Key Takeaways

1. forms.Form → custom forms
2. forms.ModelForm → forms linked to models, can save directly
3. form.is_valid() → runs validation
4. form.cleaned_data → safe, validated data
5. File uploads require request.FILES + enctype="multipart/form-data“
6. You can add custom field-level validation with clean_<fieldname>

Django forms

LAB 10: Class Work

1. Create a Contact Form with forms.Form
2. Create a Student Form using forms.ModelForm
3. Add validation to ensure the name has no numbers
4. Add a profile image upload field
5. Render all forms in Bootstrap-styled templates

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ModelForms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Django forms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Django forms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Django forms

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Django ORM queries

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Models & migrations

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Static & media files

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Templates & template inheritance

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Templates & template inheritance

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Templates & template inheritance

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

LAB 2 ACTIVITY - Login System (Authentication)

🏆 **Lab 2 – Login System (Authentication)**

Students App will support:

- ➕ Create Student
- 📄 View Students
- ✏️ Edit Student
- ❌ Delete Student
- 🔐 Login / Logout
- 🛑 Only logged-in users can access CRUD

Django project & app structure

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

PYTHON PROGRAMMING FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FRONTEND FUNDAMENTALS (HTML, CSS, JS)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DATABASES & SQL

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DJANGO CORE FUNDAMENTALS

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ADVANCED DJANGO DEVELOPMENT

ADVANCED DJANGO DEVELOPMENT

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

ADVANCED DJANGO DEVELOPMENT

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

API DEVELOPMENT (DJANGO REST FRAMEWORK)

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

VERSION CONTROL & COLLABORATION

VERSION CONTROL & COLLABORATION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

VERSION CONTROL & COLLABORATION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

SECURITY & BEST PRACTICES

SECURITY & BEST PRACTICES

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

SECURITY & BEST PRACTICES

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DEPLOYMENT & PRODUCTION

DEPLOYMENT & PRODUCTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

DEPLOYMENT & PRODUCTION

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

FINAL CAPSTONE PROJECT

FINAL CAPSTONE PROJECT

Project Options

- School ERP System
- Hospital Management System
- E-Commerce Platform
- HR Management System
- Multi-Tenant SaaS Application

Requirements

- Authentication & authorization
- Role-based access
- CRUD operations
- REST APIs
- Responsive UI
- Live deployment
- GitHub repository

FINAL CAPSTONE PROJECT

Outcome

✓ Portfolio-ready real-world application

✓ Job & internship readiness

📜 OPTIONAL CERTIFICATIONS

- Python Certification
- Django Developer Certification
- PostgreSQL Certification
- Git & GitHub Certification

FINAL CAPSTONE PROJECT

SKILLS AFTER COMPLETION

✓ Python Programming
✓ Django Web Development
✓ Frontend (HTML, CSS, JS)
✓ Database Design & ORM
✓ REST API Development
✓ Authentication & Security
✓ Deployment & DevOps Basics

CAREER PATHS

- Django Backend Developer
- Full-Stack Developer
- Python Software Engineer
- Web Application Developer
- Junior DevOps Engineer

FINAL CAPSTONE PROJECT

Web Development with Flask (Python)

WHAT IS WEB DEVELOPMENT?

Web Development is the process of creating applications that run on the internet or intranet.

A web application typically has **three main parts**:

- **Frontend (Client Side)**
  - What users see
  - **Built with:**
    - HTML (structure)
    - CSS (design)
    - JavaScript (interaction)

Capstone Project: End-to-End Web Application using Django + Database

End Of Django Web Development!